

Vocabulary Extension and Embedding Initialization for Fine-Tuning Foundational LLMs

A Hands-On Tutorial for a Term Project

Course Notes

June 6, 2026

Abstract

When we adapt a foundational language model (LLM) to a new *language* or a new *domain*, the model’s tokenizer is often the wrong tool for the job: it shatters the target text into too many tokens, wastes vocabulary slots, and inflates both model size and sequence length. The fix is to *change the vocabulary* — extend it with new tokens or replace it with a freshly trained tokenizer — and then give the new tokens sensible embedding vectors before continued training. This tutorial explains, from first principles, (i) why and when to change the vocabulary, (ii) the main *embedding initialization* methods for the new tokens (MEAN, FVT, WECHSEL, FOCUS, OFA, and baselines), (iii) the cross-lingual alignment machinery they rely on (fastText subword embeddings, the Orthogonal Procrustes problem, MUSE, CSLS), and (iv) a concrete, reproducible recipe, evaluation plan, and list of pitfalls for a term project on fine-tuning a foundational LLM.

Contents

1	Introduction and Motivation	2
2	Background: Subword Tokenization	2
2.1	Out-of-vocabulary and fallback	2
2.2	Character coverage	3
2.3	Tokenization efficiency (fertility)	3
3	Why and What to Change: Where Knowledge Lives	3
4	Problem Formulation	4
5	Embedding Initialization Methods	4
5.1	Random baselines	4
5.2	MEAN (Hewitt)	4
5.3	AVG_SUBWORDS / FVT (source-tokenizer decomposition)	5
5.4	WECHSEL (aligned bilingual fastText)	5
5.5	FOCUS (vocabulary overlap + sparsemax)	5
5.6	RAMEN and translation-based variants	6
5.7	OFA: factorized embeddings for massively multilingual init	6
6	Adaptation Regimes: Monolingual vs. Multilingual	7

7 Building Blocks: fastText, Procrustes, MUSE, CSLS	8
7.1 fastText subword embeddings	8
7.2 The Orthogonal Procrustes problem	9
7.3 MUSE: dictionaries, alignment, refinement	9
7.4 CSLS: fixing hubness	9
7.5 The distributional foundation	10
7.6 Removing the bilingual-dictionary dependency	10
8 A Recipe for the Term Project	11
8.1 Pipeline overview	11
8.2 Minimal code skeleton	11
8.3 Reference implementations	12
9 Evaluation	12
10 Pitfalls and Practical Tips	12
11 Suggested Project Milestones	13

1 Introduction and Motivation

A foundational LLM ships with a fixed **tokenizer** and a fixed **vocabulary** $\mathbb{V}$, chosen to cover its (usually English- or multilingual-)heavy pretraining data. Two situations make that vocabulary a poor fit:

Language adaptation. You want the model to work well in a target language poorly covered at pretraining (e.g. a low-resource language). The original tokenizer splits its words into many subword/byte tokens — *token inflation* — which hurts efficiency and quality.

Domain specialization / compression. You fine-tune for a narrow domain (medical, legal). A tokenizer trained *on that domain* represents it with far fewer tokens and needs far fewer vocabulary slots, shrinking the model and speeding up inference.

In both cases we (a) build a target tokenizer T^t with vocabulary $\mathbb{V}^t$, (b) give the *new* tokens embedding vectors, and (c) continue training. Step (b) — **embedding initialization** — is the crux: a good initialization preserves the knowledge already in the model and converges fast; a bad one wastes compute or destroys quality. This tutorial is built around it.

2 Background: Subword Tokenization

Modern LLMs use **subword** tokenizers (BPE, WordPiece, Unigram, usually via **SentencePiece**). They strike a balance between word-level (huge vocab, many OOVs) and character-level (tiny vocab, very long sequences) tokenization.

2.1 Out-of-vocabulary and fallback

A finite vocabulary cannot contain every possible string, so tokenizers need a **fallback** for unknown pieces. Two strategies, distinguished by the *unit* they fall back to:

	<code>character_fallback</code>	<code>byte_fallback</code>
Fallback unit	Unicode characters	UTF-8 bytes (<0xEC>...)
# fallback tokens	many (1 per kept char)	exactly 256 (all bytes)
Coverage	only chars in vocab; unseen char → <unk>	total — any string encodable, never <unk>
Cost for rare scripts	shorter if char in vocab	longer (1 char → 2–4 byte tokens)

Why `byte_fallback` for multilingual work. A vocabulary cannot store all $\sim 150,000$ Unicode characters, so `character_fallback` *will* emit <unk> for a *truly unseen character* — one absent from the tokenizer’s training corpus (e.g. a script of a language outside the training set). Since every character is a sequence of bytes and all 256 byte tokens always exist, `byte_fallback` guarantees that no string is ever lost to <unk>. The price is longer sequences for non-Latin scripts.

When does <unk> occur? The vocabulary is *frozen* after tokenizer training. <unk> is produced at *encoding time* — whenever a character not in the vocab is encountered, typically at test/inference (or during model training if its data is broader than the tokenizer’s corpus). A <unk> at inference means information is permanently lost.

2.2 Character coverage

`SentencePiece` exposes `character_coverage` (default 0.9995 for large-alphabet languages, 1.0 for small ones): the fraction of *training-corpus* characters guaranteed a token. Setting it to 1.0 keeps *every* character seen in training (incl. rare junk: typos, mojibake) — bloating the vocab and wasting slots — yet still <unk>s characters *never* seen. The modern recipe is `character_coverage<1` **plus** `byte_fallback`: a clean vocabulary *and* zero information loss.

2.3 Tokenization efficiency (fertility)

Define **fertility** as the average number of tokens per word (or per character) a tokenizer produces on a corpus. High fertility (token inflation) means longer sequences. Since attention cost grows *quadratically* in sequence length, fertility directly governs compute and is a primary reason to change the vocabulary.

3 Why and What to Change: Where Knowledge Lives

Before changing anything, understand where an LLM’s knowledge is stored:

Component	Role
Embedding matrix $\mathbf{E} \in \mathbb{R}^{ \mathbb{V} \times d}$ (+ output / LM head)	token-specific knowledge; the <i>interface</i> between tokens and the network
Transformer body (attention + FFN, all layers)	the <i>bulk</i> of knowledge: syntax, semantics, world facts, reasoning

Changing the vocabulary touches *only the embedding interface*. The transformer body — the vast majority of parameters and “understanding” — is **copied unchanged**. This is precisely why

vocabulary transfer works: we swap the dictionary, not the brain. Two compression effects follow from a smaller, domain-fit vocabulary:

1. **Fewer parameters:** the embedding/output matrix ($|\mathbb{V}| \times d$) shrinks with $|\mathbb{V}|$.
2. **Shorter sequences:** domain terms merge into single tokens (lower fertility) $\Rightarrow$ faster, cheaper inference.

What is preserved, approximated, lost. After transfer + (optional) continued training: *preserved* — the body, and embeddings of tokens shared with the old vocab; *approximated* — new tokens warm-started from old information, then refined by training; *lost* — the ability to efficiently represent old-vocabulary tokens that were dropped (general-domain breadth), which is exactly the capacity a specialized model does not need. **The original vocabulary is used only once, at initialization, and then discarded** (keeping it would erase the compression gains).

4 Problem Formulation

Let the **source** model have tokenizer T^s , vocabulary $\mathbb{V}^s$, input embeddings $\mathbf{E}^s \in \mathbb{R}^{|\mathbb{V}^s| \times d}$, and body parameters Θ . We build a **target** tokenizer T^t (trained on target language/domain text) with vocabulary $\mathbb{V}^t$. We must produce target embeddings $\mathbf{E}^t \in \mathbb{R}^{|\mathbb{V}^t| \times d}$ and reuse Θ .

Partition the target vocabulary:

$$\mathbb{O} = \mathbb{V}^s \cap \mathbb{V}^t \quad (\text{overlapping tokens, exact string match}), \quad \mathbb{A} = \mathbb{V}^t \setminus \mathbb{O} \quad (\text{new tokens}).$$

For $o \in \mathbb{O}$, the source already has a trained embedding, so almost all methods **copy** it: $\mathbf{E}^t(o) = \mathbf{E}^s(o)$. The methods differ in how they initialize $\mathbf{E}^t(a)$ for $a \in \mathbb{A}$. After initialization, copy Θ and run **continued pretraining** and/or **task fine-tuning**.

5 Embedding Initialization Methods

We present the main families, from simplest to most elaborate. A consolidated comparison is in Table 1.

5.1 Random baselines

FullRand. Train from scratch in the target language (random embeddings, random body). The reference “no transfer” baseline.

TransInner [22]. Randomly initialize $\mathbf{E}^t$ but *transfer the body* Θ from the source model. Isolates the value of reusing the body.

5.2 Mean (Hewitt)

MEAN [12] sets every new token to the **centroid of all existing source embeddings**, optionally sampled from a Gaussian fitted to their distribution:

$$\mu = \frac{1}{|\mathbb{V}^s|} \sum_i \mathbf{E}^s(i), \quad \mathbf{E}^t(a) = \mu \quad \text{or} \quad \mathbf{E}^t(a) \sim \mathcal{N}(\mu, \Sigma), \quad \Sigma = \frac{1}{|\mathbb{V}^s|} (\mathbf{E}^s - \mu)^\top (\mathbf{E}^s - \mu).$$

It uses *no* tokenizer and *no* external resources, placing new tokens in the right region (scale/location) of embedding space so the body is not shocked. The Gaussian variant spreads tokens across the real distribution instead of collapsing them to one point. **Despite its simplicity, Mean is a remarkably strong baseline** once continued pretraining is applied [23, 25]: always include it.

5.3 avg_subwords / FVT (source-tokenizer decomposition)

These initialize a new token from the pieces the **source tokenizer** produces for it. AVG_SUBWORDS [13] and Fast Vocabulary Transfer (FVT) [14, 15] copy overlaps and, for $a \in \mathbb{A}$ split by T^s into pieces $p_1, \dots, p_k$,

$$\mathbf{E}^t(a) = \frac{1}{k} \sum_{j=1}^k \mathbf{E}^s(p_j).$$

FVT was introduced for **vocabulary compression**: a tokenizer trained on the downstream domain yields a smaller vocab and lower fertility, giving up to ~15% parameter reduction and ~32% shorter sequences in-domain with almost no accuracy loss [15].

Caveat. This is the only family that *tokenizes the target token with the source tokenizer*. It works well for **same-script** extension (the source tokenizer yields meaningful subwords) but **degrades across scripts**: a foreign-script token decomposes into bytes/<unk> under the source tokenizer, making the average meaningless. That gap motivates the similarity-based methods below.

5.4 Wechsel (aligned bilingual fastText)

WECHSEL [16] targets *monolingual* source models and initializes each target token as a **semantic convex combination** of source embeddings. It works in three steps:

1. Obtain **aligned** bilingual static word embeddings W^s, W^t (Section 7.3).
2. For each token x in $\mathbb{V}^s$ and $\mathbb{V}^t$, build a *subword* embedding by summing its fastText character- n -gram vectors (Section 7.1): $u_x = \sum_{g \in G(x)} w_g$. This yields U^s, U^t in one shared space.
3. For each target token t , compute cosine similarities to all source tokens, turn them into weights $\alpha_{t,s}$ (softmax over nearest neighbours), and set

$$\mathbf{E}^t(t) = \sum_{s \in \mathbb{V}^s} \alpha_{t,s} \mathbf{E}^s(s).$$

The link between languages is *semantic* (shared aligned space), so WECHSEL works *across scripts* — unlike FVT.

5.5 Focus (vocabulary overlap + sparsemax)

FOCUS [17] targets *multilingual* source models (e.g. XLM-R, the Glot500 base) and uses the **overlap** $\mathbb{O}$ as anchors — no cross-lingual alignment required, because overlapping tokens are *identical strings* that already carry a source embedding.

1. Copy overlaps: $\mathbf{E}^t(o) = \mathbf{E}^s(o)$ for $o \in \mathbb{O}$.
2. Train an **auxiliary fastText on target text pre-tokenized with the target tokenizer**, giving a static vector for every target token (both $\mathbb{O}$ and $\mathbb{A}$).

- For each new token $a \in \mathbb{A}$, compute similarities to the overlapping tokens in that auxiliary space, apply **sparsemax** (most weights become exactly 0), and combine the overlaps’ *source* embeddings:

$$\mathbf{E}^t(a) = \sum_{o \in \mathbb{O}} w_{a,o} \mathbf{E}^s(o), \quad w_a = \text{sparsemax}(\cos(a, o))_{o \in \mathbb{O}}.$$

FOCUS never uses the source tokenizer and needs no alignment; it relies on the overlap $\mathbb{O}$ being large, which holds for multilingual sources.

5.6 Ramen and translation-based variants

RAMEN [18] initializes each target embedding as a combination of source embeddings weighted by *word-translation probabilities* (estimated from parallel data or a statistical aligner) rather than by static-embedding cosine similarity. It is a close cousin of WECHSEL, trading the aligned fastText space for explicit translation tables.

5.7 Ofa: factorized embeddings for massively multilingual init

OFA (“One For All”) [19] targets the hardest case: initializing a *huge* new vocabulary spanning *hundreds* of languages at once, for efficient large-scale continued pretraining — the Glot500-style setting. It combines two ideas.

(1) Similarity-based init from aligned multilingual vectors. Like WECHSEL/FOCUS, each unseen subword is initialized as a similarity-weighted combination of source embeddings; the similarities come from *external, well-aligned multilingual static word vectors*, so the alignment knowledge for many languages is injected at once (“one model for all”). No per-pair alignment is run — the external vectors are already multilingual.

(2) Factorized (low-rank) embedding matrix. A vocabulary covering 500+ languages would make the embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathbb{V}| \times D}$ enormous. OFA replaces it with two smaller matrices,

$$\mathbf{E} \approx \mathbf{F} \mathbf{P}, \quad \mathbf{F} \in \mathbb{R}^{|\mathbb{V}| \times d_{\text{low}}}, \quad \mathbf{P} \in \mathbb{R}^{d_{\text{low}} \times D}, \quad d_{\text{low}} \ll D,$$

where $\mathbf{F}$ holds a low-dimensional “primitive” vector per token and $\mathbf{P}$ is a *shared* up-projection to the model’s hidden size. This sharply reduces the embedding parameter count — the main bottleneck when scaling the vocabulary to many languages.

Why it matters. Fewer parameters, **faster convergence** of continued pretraining (lower compute and carbon cost), and competitive-or-better cross-lingual downstream results. OFA is the canonical *multilingual-extension* initializer (Section 6). A hypernetwork-based successor, HYPER-OFA [20], replaces the explicit similarity combination with a learned network that *predicts* unseen-token embeddings.

Decision guide.

- **Domain compression, same script:** FVT (or MEAN).
- **New language, monolingual source:** WECHSEL.
- **New language, multilingual source (XLM-R / Glot500):** FOCUS.

Method	New-token init	Src tokenizer?	External emb.?
FullRand / TransInner	random	no	no
MEAN (Hewitt)	centroid μ of all source emb. ($\pm\mathcal{N}(\mu, \Sigma)$)	no	no
AVG.SUBWORDS/FVT	mean of source-tokenizer decomposition	yes	no
WECHSEL	convex combo via aligned bilingual fastText sim	no	yes
FOCUS	sparsemax combo of overlap via target fastText sim	no	yes
RAMEN/OFA	translation/external-vector combinations	no	yes

Table 1: Embedding-initialization methods. All except the random baselines copy overlapping tokens directly; they differ in how new tokens are built.

- **Many languages at once, stay multilingual:** OFA (extension regime, Section 6).
- **Always:** include MEAN as a baseline — it is hard to beat.

6 Adaptation Regimes: Monolingual vs. Multilingual

A common misconception is that changing the vocabulary necessarily yields a *monolingual* target model. It does not. The outcome is set by **two independent design choices**. The initializers of Section 5 are reusable in *both* regimes — they only decide how new tokens get good vectors; monolinguality comes from the choices below.

Axis 1 — Replace vs. Extend the vocabulary

	Replacement	Extension
New vocabulary	fresh target tokenizer; source-only tokens dropped	keep all source tokens, append new ones
Embedding matrix	rebuilt (often smaller)	grows
Tendency	monolingual / specialized	stays multilingual
Examples	WECHSEL, FOCUS, FVT	OFA, Glot500-style

Axis 2 — Continued-training data

Even with the same vocabulary operation, *what you train on* decides whether multilinguality survives:

- **Target-only data** $\Rightarrow$ the shared transformer body drifts toward the target and **forgets** the other languages (*catastrophic forgetting*); the model becomes monolingual in effect, even if the vocabulary was merely *extended*.
- **Mixed / multilingual data** (with replay of the old languages) $\Rightarrow$ multilinguality is retained while the target is added.

The outcome matrix

Goal	Vocab op (Axis 1)	Training data (Axis 2)
Monolingual target model	replace	target-only
Domain-specialized / compressed	replace (smaller)	in-domain
Stay multilingual (+ new languages)	extend	multilingual mix (replay)

A monolingual result requires *both* replacing the vocabulary *and* training target-only; a multilingual result (e.g. Glot500) requires *extending* the vocabulary *and* training on a multilingual mixture.

Catastrophic forgetting and how to avoid it

To keep a model multilingual while adding languages:

- **Replay:** mix a proportion of the original languages’ data into continued pretraining (the route Glot500 [26] and OFA [19] take).
- **Parameter-efficient adaptation:** freeze the pretrained weights and train small added modules — adapters or **LoRA** [21] — so the original capability is protected by construction.
- **Factorized / low-rank embeddings** (OFA) keep the added parameters small, easing the joint multilingual optimization.

Takeaway. The initializers (MEAN, FVT, WECHSEL, FOCUS, OFA) are *orthogonal* to the regime: pick the initializer for how new tokens get vectors, and pick the regime (Axes 1–2) for whether the result is monolingual or multilingual.

7 Building Blocks: fastText, Procrustes, MUSE, CSLS

7.1 fastText subword embeddings

fastText [1] represents a word/token as a **bag of character n -grams**. Wrap the token with boundary markers <,>, then extract *all* n -grams for $n \in [\text{minn}, \text{maxn}]$ (defaults 3 and 6; minn, maxn are fixed hyperparameters, n ranges over the whole band), plus the whole-token unit. The token vector is the sum:

$$u_w = \sum_{n=\text{minn}}^{\text{maxn}} \sum_{g \in n\text{-grams}_n(w)} z_g.$$

Only the n -gram vectors z_g are *learned* (by skip-gram/CBOW); the n -gram *set* is fixed by the hyperparameters. This is why *any* token, even an unseen one, can be vectorized — a property WECHSEL and FOCUS rely on.

Hashing and collisions. To bound memory, n -grams are hashed into a fixed number of *buckets* (default 2,000,000); each bucket holds *one* vector, so many n -grams may **share** a vector (collision). Coverage (a vector always exists) is guaranteed by hashing; *quality* is not — an unseen n -gram’s bucket reflects only whatever collided there. The summation over many n -grams per token dilutes this noise. (Real words get private, collision-free rows; only n -grams are hashed.)

7.2 The Orthogonal Procrustes problem

To compare source and target embeddings, they must share one space. Given matched **anchor** pairs (dictionary translations) stacked as columns $A = [a_1, \dots, a_n]$ (source) and $B = [b_1, \dots, b_n]$ (target), find the rotation that best maps one onto the other:

$$\Omega^* = \arg \min_{\Omega^\top \Omega = I} \|\Omega A - B\|_F^2.$$

Why orthogonal. An orthogonal map is an *isometry*: it preserves all inner products, distances, angles, and cosines, so it re-orientes the source space onto the target *without distorting* each language’s internal geometry. (General-linear maps shear/scale and overfit; nonlinear maps distort.)

Closed-form solution. With $W^\top W = I$, minimizing $\|\Omega A - B\|_F^2$ is equivalent to maximizing $\text{tr}(\Omega^\top M)$ with the cross-covariance $M = BA^\top$. Taking the SVD $M = U\Sigma V^\top$,

$$\boxed{\Omega^* = UV^\top} \quad (\text{Schönemann [6]}).$$

One SVD, global optimum, no iteration. Apply Ω^* to *all* source vectors to land both languages in one space.

Why a *single* rotation (first principles). “Same relational structure” means the two point clouds are **congruent**: $\langle a_i, a_j \rangle = \langle b_i, b_j \rangle$ for translations. A map preserving all inner products (origin-fixed) is provably a single linear orthogonal transformation; a rigid body has one orientation, so one global Ω must work everywhere. A per-word map would be the dictionary itself (no generalization). The estimate fit on a few thousand anchors generalizes to the whole vocabulary because embedding spaces are **approximately isometric** across languages (shared distributional structure), so Ω^* correctly places even non-anchor words.

Why it generalizes / its limits. The isometry hypothesis is grounded in a (largely) shared world, shared cognition, and shared communication, mediated by *distribution* not direct reference. It is only *approximate*: lexical gaps, polysemy, cultural drift, and typological distance break it, and empirically isomorphism degrades for distant or low-resource pairs [8, 9]. The nonzero Procrustes residual measures this imperfection.

7.3 MUSE: dictionaries, alignment, refinement

MUSE [7] provides bilingual dictionaries and alignment tooling. **Supervised:** build anchors from a dictionary (e.g. $\sim 5k$ pairs) and solve Procrustes (Section 7.2); this is what WECHSEL uses. **Unsupervised:** learn Ω by *adversarial* training (a discriminator tries to tell ΩA from B), then **refine** via Procrustes on a synthetic dictionary of mutual nearest neighbours. Preprocessing (unit-normalization, mean-centering) makes the Frobenius objective behave like a cosine objective.

7.4 CSLS: fixing hubness

In high dimensions, some target vectors are nearest neighbours of *many* sources (the **hubness** problem), corrupting retrieval. *Cross-domain Similarity Local Scaling* (CSLS) corrects this by subtracting each point’s mean neighbourhood similarity:

$$\text{CSLS}(\Omega x_s, y_t) = 2 \cos(\Omega x_s, y_t) - r_T(\Omega x_s) - r_S(y_t), \quad r_T(z) = \frac{1}{K} \sum_{y \in \mathcal{N}_T(z)} \cos(z, y).$$

Use CSLS (not raw cosine) for nearest-neighbour lookups in the aligned space.

7.5 The distributional foundation

All of the above rests on the **distributional hypothesis** [10, 11]: meaning is captured by co-occurrence. Embeddings encode meaning as relative position; cross-lingual methods exploit the fact that this *relative* structure is shared across languages.

7.6 Removing the bilingual-dictionary dependency

Supervised Procrustes (Section 7.2) needs a bilingual dictionary as anchors. For a well-resourced pair this is a cheap, *pre-built* public resource — e.g. MUSE ships dictionaries *and* even pre-aligned fastText for ~ 45 languages (incl. Greek, Hebrew, Korean), so the practitioner just downloads them. The dependency only bites for the *uncovered* long tail. Three strategies remove it.

1. Manufacture the anchors for free (unsupervised alignment). The dictionary can be replaced by anchors that cost nothing:

- **Identical-string seed.** Words spelled identically in both languages (numerals, proper names, cognates, shared punctuation) form a free anchor set; run Procrustes + iterative refinement on it. MUSE supports this via `--dico.train identical.char`. (Needs a shared script.)
- **Adversarial alignment** [7]. Learn Ω with a discriminator that tries to separate ΩA from B — using *no* anchors — then **refine** via Procrustes on a synthetic dictionary of mutual nearest neighbours the model generates itself. Robust self-learning variants [5] reach the same with no seed. The “dictionary” becomes *self-generated*.

2. Use the shared vocabulary as anchors (Focus). FOCUS [17] avoids alignment *entirely*. A multilingual source already shares many *identical token strings* with the target tokenizer, and those overlapping tokens already carry trained source embeddings — so they *are* the anchors: no dictionary, no Procrustes. Similarity for the remaining new tokens comes from a fastText trained on the target data itself. The requirement shifts from “have a dictionary” to “have large vocabulary overlap” (which holds for multilingual sources).

3. Inherit a once-built alignment (Ofa). OFA [19] reuses *pre-aligned* multilingual vectors (ColexNet+) that already span 1,335 languages in a single space. The alignment/dictionary cost was paid *once*, by the resource’s creators (from a massively parallel corpus via colexification), and is shared. The requirement shifts from “build a dictionary” to “the target language is covered by the resource.”

Common thread. Every route either **manufactures anchors for free** (identical strings, adversarial self-learning, or vocabulary overlap) or **amortizes the cost** of building them across the community (a once-built, shared, pre-aligned resource). The supervised dictionary is therefore a *convenience* for covered languages, not a hard requirement — which is why the dictionary-free methods (FOCUS, OFA) are the ones that scale to the low-resource tail.

8 A Recipe for the Term Project

8.1 Pipeline overview

1. **Pick a source model** matched to the setting: monolingual (e.g. an English model) for WECHSEL, or multilingual (XLM-R, Glot500) for FOCUS.
2. **Collect target data:** a raw, unannotated, monolingual corpus in the target language/domain (e.g. Wikipedia or web crawl). No labels or parallel data are required.
3. **Train the target tokenizer** on that corpus with `SentencePiece` (set `character_coverage<1` and `byte_fallback=True`); choose the vocab size (a compression knob).
4. **Compute the overlap** $\mathbb{O} = \mathbb{V}^s \cap \mathbb{V}^t$ and the new set $\mathbb{A}$.
5. **Initialize E^t** with a chosen method (Section 5); copy overlaps, build new tokens, copy the body Θ .
6. **Continued pretraining** (MLM/CLM) on the target corpus to adapt the new embeddings and re-align the body; then **task fine-tuning**.
7. **Evaluate** intrinsically and on downstream tasks (Section 9).

8.2 Minimal code skeleton

```
# 1. Train a target tokenizer (SentencePiece via HF tokenizers or spm)
import sentencepiece as spm
spm.SentencePieceTrainer.train(
    input="target_corpus.txt", model_prefix="tgt", vocab_size=32000,
    character_coverage=0.9995, byte_fallback=True, model_type="unigram")

# 2. Load source model + tokenizer (HuggingFace)
from transformers import AutoModelForMaskedLM, AutoTokenizer
src_tok = AutoTokenizer.from_pretrained("xlm-roberta-base")
model = AutoModelForMaskedLM.from_pretrained("xlm-roberta-base")
E_s = model.get_input_embeddings().weight.data # (/V_s/, d)

# 3. Build target vocab, overlap, and new embedding matrix E_t
tgt_vocab = load_tgt_vocab("tgt.vocab")
overlap = [t for t in tgt_vocab if t in src_tok.get_vocab()]
new_toks = [t for t in tgt_vocab if t not in src_tok.get_vocab()]
import torch
E_t = torch.empty(len(tgt_vocab), E_s.shape[1])

# 3a. Copy overlapping tokens
for t in overlap:
    E_t[tgt_id[t]] = E_s[src_tok.convert_tokens_to_ids(t)]

# 3b. Initialize new tokens -- pick ONE method:
# MEAN: E_t[i] = E_s.mean(0) (+ optional Gaussian noise)
# FVT: E_t[i] = E_s[src_tok.encode(token_str)].mean(0)
# WECHSEL/FOCUS: weighted combo via fastText similarity (see their repos)

# 4. Resize model embeddings and load E_t; then continued pretraining.
```

```
model.resize_token_embeddings(len(tgt_vocab))
model.get_input_embeddings().weight.data.copy_(E_t)
```

8.3 Reference implementations

- MUSE (alignment, dictionaries, CSLS): <https://github.com/facebookresearch/MUSE>
- FOCUS: <https://github.com/konstantinjdobler/focus>
- WECHSEL: <https://github.com/CPJKU/wechsel>
- HuggingFace `transformers/tokenizers`, `sentencepiece`, and the official `fastText` library.

9 Evaluation

Evaluate at three levels and *ablate the initialization method* (always include MEAN and FULLRAND as baselines).

Intrinsic (tokenizer / coverage).

- **Fertility** (tokens/word) on held-out target text — lower is better.
- **Overlap size** $|\mathbb{O}|/|\mathbb{V}^t|$ — how many tokens are copied directly.
- **<unk> rate / byte-fallback rate** — coverage of the new tokenizer.

Initialization quality (before continued training). Measure zero-/few-step loss on target text right after initialization, and nearest-neighbour sanity checks (does a new token sit near sensible source tokens?). A good init starts with lower loss and converges faster.

Downstream. Task metrics (accuracy/F1 for classification; perplexity for LM; BLEU/chrF for generation) after continued pretraining + fine-tuning. Report against compute budget (the fair axis: quality vs. GPU-hours).

Efficiency (for compression projects). Parameter count, average sequence length / fertility, and inference latency vs. the original model, alongside the accuracy trade-off.

10 Pitfalls and Practical Tips

- **Always enable** `byte_fallback` for multilingual/low-resource targets, so no script collapses to `<unk>`.
- **Mean is a strong baseline.** Fancy methods often beat it by less than the evaluation noise once continued pretraining is applied [23, 25]. Do not claim a win without it.
- **Match method to setting:** FVT (same script / compression), WECHSEL (monolingual source), FOCUS (multilingual source).
- **Cross-lingual alignment is approximate** and degrades for typologically distant or low-resource pairs [8, 9]; expect weaker initialization there.

- **Use CSLS**, not raw cosine, for nearest-neighbour lookups in the aligned space (hubness).
- **Specialization trade-off**: dropping vocabulary loses general-domain breadth. Fine for a task/domain model; harmful for a general-purpose generative LLM.
- **Continued pretraining matters**: initialization is a *warm start*; most of the recovery comes from training on target data. Budget for it.
- **Discard the old vocabulary** after initialization (keeping it erases compression gains); the new tokenizer is self-sufficient.

11 Suggested Project Milestones

1. Choose a source model + target language/domain; collect and clean a monolingual corpus.
2. Train the target tokenizer; report fertility vs. the source tokenizer and overlap statistics.
3. Implement **three** initializations (FULLRAND or TRANSINNER, MEAN, and one of FVT/WECHSEL/FOCUS).
4. Continued pretraining under a fixed compute budget; track loss curves per method.
5. Fine-tune and evaluate on a downstream task; ablate the initialization.
6. (Optional) Compression variant: shrink the vocab, measure size/latency vs. accuracy.
7. Write up: which method won, by how much, and *was it within noise of* MEAN?

References

- [1] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov. *Enriching Word Vectors with Subword Information*. TACL, 2017. (fastText.)
- [2] T. Mikolov, Q. V. Le, I. Sutskever. *Exploiting Similarities among Languages for Machine Translation*. arXiv:1309.4168, 2013.
- [3] C. Xing, D. Wang, C. Liu, Y. Lin. *Normalized Word Embedding and Orthogonal Transform for Bilingual Word Translation*. NAACL, 2015.
- [4] M. Artetxe, G. Labaka, E. Agirre. *Learning principled bilingual mappings of word embeddings while preserving monolingual invariance*. EMNLP, 2016.
- [5] M. Artetxe, G. Labaka, E. Agirre. *A robust self-learning method for fully unsupervised cross-lingual mappings*. ACL, 2018.
- [6] P. H. Schönemann. *A generalized solution of the orthogonal Procrustes problem*. Psychometrika, 31(1):1–10, 1966.
- [7] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, H. Jégou. *Word Translation Without Parallel Data*. ICLR, 2018. (MUSE; arXiv:1710.04087.)
- [8] A. Søgaard, S. Ruder, I. Vulić. *On the Limitations of Unsupervised Bilingual Dictionary Induction*. ACL, 2018.

- [9] I. Vulić, S. Ruder, A. Søgaard. *Are All Good Word Vector Spaces Isomorphic?* EMNLP, 2020.
- [10] Z. Harris. *Distributional Structure*. Word, 10(2–3):146–162, 1954.
- [11] J. R. Firth. *A Synopsis of Linguistic Theory 1930–1955*. Studies in Linguistic Analysis, 1957.
- [12] J. Hewitt. *Initializing New Word Embeddings for Pretrained Language Models*. 2021. <https://www.cs.columbia.edu/~johnhew/vocab-expansion.html> (MEAN; centroid μ / $\mathcal{N}(\mu, \Sigma)$.)
- [13] (See [12]; the `avg_subwords` decomposition variant is commonly attributed in surveys but is realized in FVT [14].)
- [14] L. Gee, A. Zugarini, L. Rigutini, P. Torrioni. *Fast Vocabulary Transfer for Language Model Compression*. EMNLP (Industry Track), 2022.
- [15] L. Gee, et al. *Fast Vocabulary Transfer for Language Model Compression*. arXiv:2402.09977, 2024. (Extended results: compression, fertility, speedups.)
- [16] B. Minixhofer, F. Paischer, N. Rekasaz. *WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models*. NAACL, 2022. (arXiv:2112.06598.)
- [17] K. Dobler, G. de Melo. *FOCUS: Effective Embedding Initialization for Monolingual Specialization of Multilingual Models*. EMNLP, 2023. (arXiv:2305.14481.)
- [18] K. Tran. *From English to Foreign Languages: Transferring Pre-trained Language Models*. arXiv:2002.07306, 2020. (RAMEN.)
- [19] Y. Liu, P. Lin, M. Wang, H. Schütze. *OFA: A Framework of Initializing Unseen Subword Embeddings for Efficient Large-scale Multilingual Continued Pretraining*. Findings of NAACL, 2024. (arXiv:2311.08849; code: <https://github.com/cisnlp/ofa>.)
- [20] *HyperOFA: Expanding LLM Vocabulary to New Languages via Hypernetwork-Based Embedding Initialization*. arXiv:2504.21018, 2025.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [22] W. de Vries, M. Nissim. *As Good as New. How to Successfully Recycle English GPT-2 to Make Models for Other Languages*. Findings of ACL, 2021. (TRANSINNER.)
- [23] *An Empirical Comparison of Vocabulary Expansion and Initialization Approaches for Language Models*. arXiv:2407.05841, 2024.
- [24] A. Yamaguchi, et al. *An Empirical Study on Cross-lingual Vocabulary Adaptation for Efficient Language Model Inference*. arXiv:2402.10712, 2024.
- [25] *SambaLingo: Teaching Large Language Models New Languages*. arXiv:2404.05829, 2024.
- [26] A. ImaniGooghari, et al. *Glott500: Scaling Multilingual Corpora and Language Models to 500 Languages*. ACL, 2023.